

## Análise e Desenvolvimento de Sistemas

# Gerenciamento Ágil de Projetos

---

## DDD: entendendo o domínio do problema

---

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

### 1. Domínio (Domain)

É a **razão de ser** do software. Refere-se à área de negócio da empresa, ao problema real que precisa ser resolvido ou à atividade que está sendo automatizada.

- **O que é:** O ecossistema do cliente (ex: logística, finanças, saúde).
- **Foco:** Entender como as regras funcionam no mundo real antes de escrever o código.
- **Divisão:** É quebrado em subdomínios (Principal, Suporte e Genérico) para facilitar o gerenciamento do sistema.

### 2. Linguagem Ubíqua (Ubiquitous Language)

É o **idioma comum** falado por todas as pessoas envolvidas no projeto. Ela une os desenvolvedores e os especialistas do negócio (Product Owners, diretores, usuários).

- **O que é:** Um vocabulário único e livre de jargões técnicos complexos.
- **Aplicação:** Os mesmos termos discutidos nas reuniões devem aparecer nos diagramas, nas tarefas e, principalmente, no **código-fonte** (nomes de classes, funções e variáveis).
- **Objetivo:** Eliminar falhas de comunicação e traduções erradas entre o negócio e a TI.

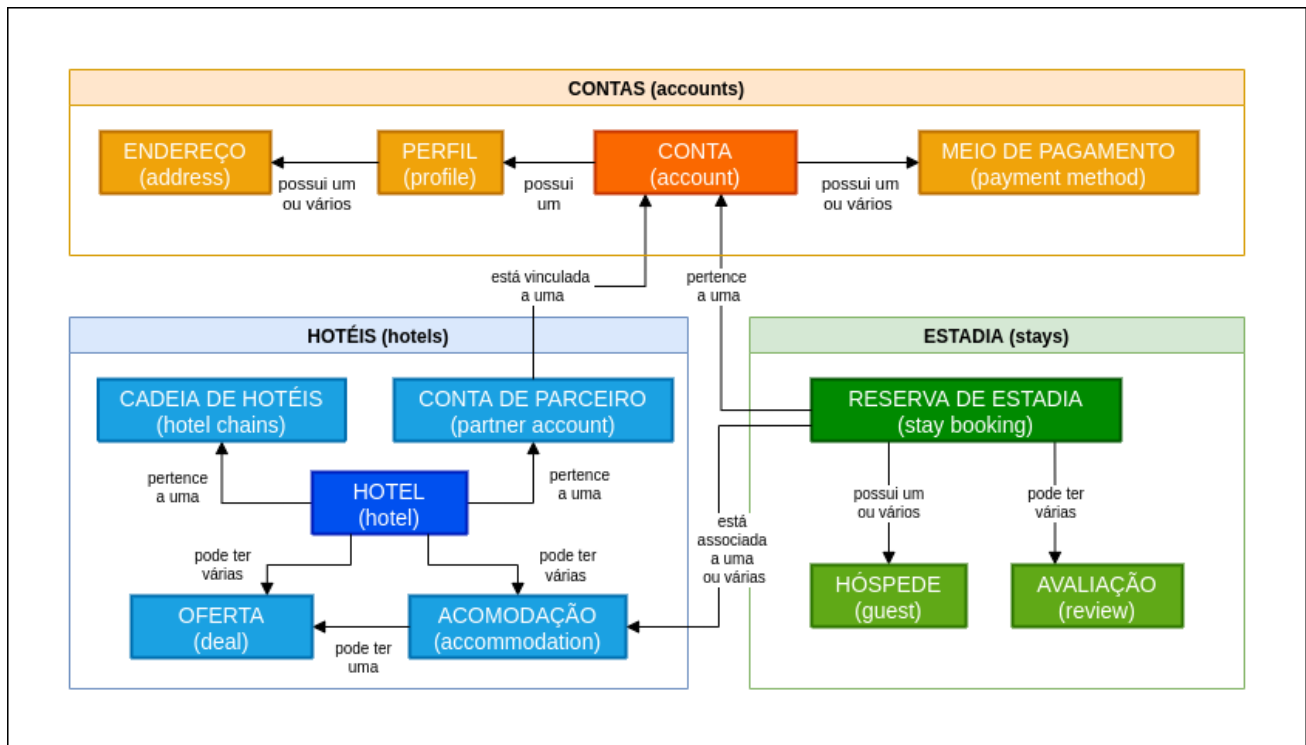
### 3. Contexto Delimitado (Bounded Context)

É a **fronteira lógica** onde um modelo de domínio e sua Linguagem Ubíqua se aplicam. O mesmo termo pode ter significados totalmente diferentes dependendo do contexto.

- **O que é:** Uma linha divisória que protege o significado dos conceitos.
- **Exemplo Clássico:** A palavra "Produto".
  - No contexto de **Vendas**, "Produto" tem preço, estoque e descrição.
  - No contexto de **Entrega/Logística**, o mesmo "Produto" vira um item com peso, altura, largura e código de rastreio.
- **Objetivo:** Evitar modelos de dados gigantescos e confusos, permitindo que cada equipe foque em sua regra de negócio específica.

---

## GLOSSÁRIO DO DOMÍNIO DE UM SISTEMA:



## 1. Entendendo o Domínio: O Ponto de Partida

Antes de criar qualquer classe, o modelador escuta os especialistas de negócio. Nesse processo, busca-se entender:

- Os fluxos de processo: O que acontece primeiro? O que engatilha uma ação?
- As regras de negócio (Invariantes): O que *nunca* pode acontecer ou o que deve ser *sempre* verdade no sistema?

A partir dessa massa de conhecimento, começamos a segmentar o domínio utilizando os padrões táticos do DDD.

## 2. Como identificar e modelar Entidades

Para identificar se um conceito do mundo real deve ser modelado como uma Entidade, o modelador deve fazer a seguinte pergunta: "Se os atributos deste objeto mudarem completamente, ele ainda continua sendo a mesma coisa?"

- Critério de Identificação: A característica essencial de uma Entidade é a sua Identidade Única. Ela tem um ciclo de vida: ela nasce, é modificada e pode ser destruída, mas seu ID permanece inalterado.
- Como funciona a modelagem: \* A igualdade entre duas entidades é determinada exclusivamente pelos seus IDs, e não pelos seus valores.
  - Elas contêm comportamento (métodos que alteram seu estado interno) e garantem que suas propriedades estejam sempre válidas após qualquer modificação.

## 3. Como identificar e modelar Value Objects (Objetos de Valor)

Para identificar um Value Object (VO), a pergunta inverte-se: "Se eu alterar o valor de um atributo deste objeto, ele se torna conceitualmente outra coisa?" ou "Eu me importo com *quem* este objeto é, ou apenas com o que ele *contém*?"

- Critério de Identificação: VOs não possuem identidade própria. Eles servem apenas para descrever, quantificar ou qualificar uma característica de uma Entidade. Se dois VOs possuem os mesmos valores exatos em todos os campos, eles são considerados idênticos e intercambiáveis.
- Como funciona a modelagem:
  - Imutabilidade: Uma vez criado, um VO nunca muda. Se você precisar alterar um endereço ou uma quantia monetária, você descarta o objeto antigo e cria uma nova instância com os novos valores.
  - Validação na Criação: Um VO deve validar a si mesmo no momento em que é instanciado (seu construtor impede a criação de um estado inválido).

#### 4. Como identificar e modelar Agregados (e Raízes de Agregação)

A identificação de um Agregado é uma das tarefas mais complexas do DDD. O modelador deve olhar para o grupo de Entidades e Value Objects identificados e perguntar: "Quais desses objetos precisam ser modificados juntos para que nenhuma regra de negócio seja quebrada (consistência transacional)?"

- Critério de Identificação: Um Agregado é uma fronteira conceitual que agrupa objetos relacionados. Dentro dele, escolhe-se uma única Entidade para ser a Raiz de Agregação (Aggregate Root).
- Como funciona a modelagem:
  - Portão de Entrada: O mundo externo ao Agregado *só pode falar com a Raiz*. Nenhuma classe externa pode referenciar diretamente ou alterar uma entidade interna do agregado.
  - Garantia de Regras (Invariantes): A Raiz é a única responsável por fazer cumprir as regras de negócio de todo o grupo. Se algo lá dentro precisa mudar, a ordem é dada à Raiz, e ela coordena as mudanças internas garantindo que tudo continue consistente ao final da transação.

#### 5. Como identificar e modelar Repositórios

O Repositório surge da necessidade de gerenciar o ciclo de vida dos nossos objetos de domínio (carregar do banco de dados para a memória e salvar as alterações de volta). A regra de identificação aqui é matemática e rígida: "Apenas Raízes de Agregação possuem Repositórios."

- Critério de Identificação: Se um conceito foi classificado como uma Raiz de Agregação, ele obrigatoriamente precisará de um Repositório para que o sistema possa buscá-lo ou persisti-lo. Se for uma entidade interna de um agregado ou um Value Object, ele nunca terá um repositório próprio.
- Como funciona a modelagem:
  - O Repositório funciona como uma ilusão de uma coleção em memória (como se fosse uma lista gigante de objetos daquela Raiz).
  - A camada de aplicação pede ao repositório a Raiz do Agregado inteira. Quando o Agregado é salvo, o repositório persiste a Raiz e todos os seus objetos internos de forma atômica (em uma única transação), escondendo a complexidade de tabelas SQL ou coleções NoSQL do coração do software.

#### Resumo do Fluxo de Modelagem:

1. Escuta-se o negócio para entender o comportamento.
2. Isolam-se as características imutáveis e descritivas em Value Objects.
3. Identificam-se os conceitos que possuem ciclo de vida e identidade como Entidades.
4. Agrupam-se essas Entidades e VOs sob uma fronteira de consistência chamada Agregado, elegendo uma Raiz.
5. Cria-se um Repositório exclusivo para dar acesso global a essa Raiz.